

حداقل نیازمندی‌ها و مشکلات احتمالی راه‌اندازی اکسپلوریا روی سرور واقعی

راهنمای تصمیم‌گیری برای استقرار آینده Laravel + React Monolith، با تمرکز بر زیرساخت، امنیت، پایگاه داده، سرویس‌های بیرونی و عملیات پایدار.

توضیح مهم: این سند صرفاً برای برنامه‌ریزی آینده تهیه شده است و به معنای شروع استقرار، خرید سرور یا انتشار عمومی سامانه نیست.

تاریخ تهیه

2026-08-01

نسخه سند

v1.0

وضعیت

راهنمای پیش از استقرار

معماری مینا

Laravel + React Monolith

۱. جمع‌بندی مدیریتی و وضعیت فعلی

۲۵۸ تست موفق

۴۶۵۲ Assertion بدون شکست

تحلیل ایستا سبز

PHPStan صفر خطا، TypeScript و ESLint موفق

Build موفق

فرانت‌اند Vite با ۳۳۳۹ ماژول ساخته شده است

دمو آماده

۱۹ کنترل Demo Readiness از ۱۹ کنترل موفق

نتیجه فنی: Codebase فعلی خوانا، قابل Build و از نظر مسیرهای تست‌شده عملیاتی است. معماری یکپارچه React + Laravel برای استقرار واقعی مناسب است و مانع معماری شناخته‌شده‌ای وجود ندارد.

محدودیت این نتیجه: موفقیت Local و تست خودکار، تضمین صددرصد Production نیست. رفتار PostgreSQL واقعی، HTTPS، پیامک، شبکه، مجوز فایل، فشار هم‌زمان کاربران و Backup/Restore باید در Staging مشابه سرور اصلی آزمایش شوند.

وضعیت Gate محیط فعلی

بخش	وضعیت	توضیح
دمو و داده پایه	آماده	تمام کنترل‌های دمو، QR، کمپین، مأموریت، پاداش و پنل‌ها موفق‌اند.
کد و کیفیت	آماده	تست، Build، قالب‌بندی و تحلیل ایستا موفق‌اند.
محیط Production	نیازمند تنظیم	محیط فعلی عمده Local است: OTP، SQLite، HTTP ثابت و تنظیمات توسعه.
استقرار عمومی	آینده	ابتدا Staging، سپس UAT و تأیید رسمی برای انتشار لازم است.

پاسخ کوتاه به پرسش «آیا کار می‌کند؟»

بله، در محدوده تست‌شده و محیط محلی کار می‌کند. برای ادعای آمادگی کامل سرور واقعی باید Gate استقرار، تست PostgreSQL، سرویس OTP واقعی و آزمون عملیاتی روی Staging نیز موفق شوند.

۲. حداقل زیرساخت پیشنهادی

مؤلفه	حداقل قابل قبول	پیشنهاد برای بهره‌برداری پایدار
سیستم‌عامل	Linux Server پشتیبانی‌شده	Ubuntu LTS یا Debian پایدار، به‌روز و سخت‌سازی‌شده
پردازنده	۲ vCPU برای شروع محدود	۴ vCPU با امکان افزایش پس از Load Test
حافظه	۴ GB RAM	۸ GB RAM برای PostgreSQL، PHP-FPM و Worker
فضای ذخیره‌سازی	۳۰ GB SSD	۶۰ GB یا بیشتر، به‌علاوه فضای Backup مستقل
PHP	PHP >= 8.4.1	PHP-FPM با OPcache، محدودیت حافظه مناسب و Patch امنیتی جاری
پایگاه داده	PostgreSQL پشتیبانی‌شده	سرویس مجزا یا Managed Database با Backup و مانیتورینگ
وب‌سرور	Apache یا Nginx	Nginx + PHP-FPM با Document Root روی پوشه public
TLS/HTTPS	گواهی معتبر	تمدید خودکار، HSTS پس از تثبیت، Redirect کامل HTTP به HTTPS
صف و Session	Database	Redis برای Queue/Cache/Session در بار بالاتر
ساخت فرانت‌اند	Node.js و npm هنگام Build	Build در CI؛ انتقال Artifact نهایی به سرور

افزونه‌ها و ابزارهای لازم

ابزارهای عملیاتی

- ✓ Composer 2
- ✓ tar و git، curl
- ✓ pg_dump، psql و pg_restore
- ✓ Supervisor یا systemd برای Worker
- ✓ Log rotation و مانیتورینگ منابع

PHP Extensions

- ✓ PDO و pdo_pgsql
- ✓ Session و OpenSSL، Fileinfo
- ✓ Tokenizer و DOM، XML، ctype
- ✓ Intl و Mbstring برای پردازش بهتر متن و Locale
- ✓ Zip و Curl برای Composer و ارتباط بیرونی

ریسک هاست اشتراکی: اگر PHP 8.4.1، دسترسی PostgreSQL، Worker، تنظیم Document Root و اجرای فرمان‌های Artisan در دسترس نباشد، هاست اشتراکی گزینه مناسبی برای این پروژه نیست.

۳. تنظیمات ضروری محیط و امنیت

تنظیم	مقدار/شرط	علت
APP_ENV	production	فعالسازی رفتار مناسب محیط واقعی
APP_DEBUG	false	جلوگیری از افشای SQL، Stack Trace و مسیرهای سرور
APP_URL	دامنه HTTPS واقعی	تولید لینک، Cookie و Redirect صحیح
APP_KEY	کلید یکتا و خارج از Git	رمزنگاری Session و داده‌های حساس؛ تغییر آن داده قبلی را غیرقابل خواندن می‌کند
DB_CONNECTION	pgsql	موتور پایگاه داده مصوب برای Staging/Production
QUEUE_CONNECTION	redis یا database	پایداری عملیات پس‌زمینه و Retry
SESSION_DRIVER	redis یا database	جلوگیری از گم‌شدن Session بین Process ها یا Release ها
Secure Cookie	Secure + HttpOnly	کاهش سرقت Session در شبکه و JavaScript
OTP_DRIVER	Provider واقعی	کد ثابت Local برای اینترنت ممنوع است
Logging و Mail	SMTP و Log واقعی	ارسال اعلان، تشخیص خطا و Audit عملیاتی

Secrets و دسترسی‌ها

- ✓ فایل `.env` نباید Commit یا از Web Server قابل دانلود باشد.
- ✓ توکن OTP، رمز SMTP، PostgreSQL و کلیدهای Cloud باید در Secret Store یا Environment نگهداری شوند.
- ✓ کاربر اجرای برنامه نباید root باشد و فقط به مسیرهای لازم دسترسی نوشتن داشته باشد.
- ✓ ورودی وب‌سرور باید فقط به پوشه `public` اشاره کند.
- ✓ پورت PostgreSQL و Redis نباید بدون محدودیت در اینترنت باز باشند.

سرویس‌های بیرونی که باید پیش از انتشار واقعی شوند

Email

دامنه فرستنده، SMTP، SPF/DKIM، صف ارسال و مدیریت Bounce باید مشخص شوند.

OTP/SMS

Endpoint، Token، Sender، Timeout، Retry، Rate Limit، ثبت خطا و سناریوی قطعی Provider باید آزموده شوند.

Monitoring

Health Check، خطاهای 5xx، مصرف دیسک، کندی DB و شکست Worker باید Alert داشته باشند.

Storage

فایل تبلیغاتی باید در Storage پایدار یا Object Storage قرار گیرد و با تعویض Release حذف نشود.

۴. مسیر استقرار امن و قابل بازگشت

الگوی زیر با اسکریپت موجود `scripts/deploy-staging.sh` همراستاست و باید ابتدا در Staging تمرین شود.

1 قفل نسخه و Review

Commit مشخص، Worktree تمیز، Quality Gate سبز و تأیید نسخه قابل استقرار.

2 Backup معتبر

گرفتن PostgreSQL Archive، کنترل با `pg_restore` و اطمینان از تازگی Backup.

3 ساخت Release مستقل

استخراج Commit در پوشه Release جدید؛ اتصال `env` و `storage` اشتراکی.

4 نصب وابستگی و Build

اجرای Composer بدون `npm ci`، `dev` و Build نهایی Vite.

5 پاکسازی و آماده‌سازی Laravel

پاکسازی Cache قدیمی، `storage:link` و بررسی دسترسی فایل‌ها.

6 Migration و Maintenance

ورود کنترل‌شده به Maintenance، اجرای Migration با `force` و زمان‌بندی برای کمترین وقفه.

7 Readiness Gate

اجرای `production-readiness` و `demo-readiness`؛ توقف انتشار در صورت هر Fail.

8 تعویض Atomic و Health Check

تغییر Restart، Symlink، صف، خروج از Maintenance و کنترل `HTTPS /up`.

9 Rollback در خطا

برگشت Symlink به Release قبلی، فعال‌سازی مجدد برنامه و حفظ Release ناموفق برای بررسی.

نکته: اسکریپت فعلی برای Staging نوشته شده است. نسخه Production باید جداگانه، با مسیر، دامنه، Backup Policy، Retention و مسئول تأیید مشخص تهیه شود.

۵. ریسک‌های محتمل - زیرساخت و داده

ریسک	سطح	پیامد	اقدام پیشگیرانه
PHP قدیمی یا Extension ناقص	زیاد	خطای 500، Composer یا اجرا نشدن برنامه	کنترل Platform Requirements قبل از Deploy
تفاوت SQLite و PostgreSQL	زیاد	خطای Query، Migration یا Constraint	اجرای تست و Migration روی PostgreSQL در Staging
Migration سنگین	زیاد	قفل جدول یا وقفه سرویس	Backup Plan، برآورد زمان، Rollback و Maintenance Plan
Backup غیرقابل‌بازایی	زیاد	از دست رفتن داده در خطا	Restore Drill دوره‌ای، نه فقط گرفتن فایل Backup
مجوز نادرست فایل	متوسط	خطای Log، Cache یا Upload	مالکیت محدود و نوشتن فقط در storage و cache
حذف Upload در Release	زیاد	گم‌شدن فایل تبلیغات	Shared Storage یا Object Storage و Backup مستقل
پر شدن دیسک	متوسط	توقف Log، DB، Upload	Alert، Retention، Log Rotation و ظرفیت آزاد
کمبود RAM/CPU	متوسط	Timeout و کندی شدید	Load Test، تنظیم PHP-FPM و Scaling Plan
تنظیم اشتباه Nginx	زیاد	افشای فایل یا Route‌های 404	Document Root روی public و Deny فایل‌های مخفی
Cache تنظیمات قدیمی	متوسط	استفاده از Secret یا URL قبلی	پاک‌سازی Cache قبل از optimize و Health Check

نکات پایگاه داده

- ✓ اتصال SSL به PostgreSQL، محدودیت IP و کاربر با حداقل دسترسی تعریف شود.
- ✓ Backup روزانه، نگهداری چندنسخه‌ای و یک نسخه خارج از همان سرور در نظر گرفته شود.
- ✓ شاخص‌ها، Query‌های کند و رشد جدول‌های Scan/Event پس از Load Test پایش شوند.
- ✓ ساعت سرور، PostgreSQL و Laravel با Timezone مصوب هماهنگ باشند.

اولویت عملی: مهم‌ترین تست فنی پیش از سرور اصلی، اجرای کامل Migration و Test Suite روی PostgreSQL در Staging است.

۶. ریسک‌های محتمل - امنیت و عملیات

ریسک	سطح	پیامد	اقدام پیشگیرانه
فعال بودن Debug	زیاد	افشای Stack Trace و اطلاعات فنی	APP_DEBUG=false و صفحه خطای عمومی
HTTPS یا Cookie نادرست	زیاد	اختلال ورود یا سرقت Session	TLS معتبر، Secure/HttpOnly و تست پشت Proxy
OTP ثابت یا Provider ضعیف	زیاد	دورزدن ورود یا عدم تحویل کد	Provider واقعی، Timeout، Rate Limit و Audit
Worker متوقف	متوسط	انباشت Job و تأخیر اعلان	Supervisor/systemd، Restart و Alert صف
Secret در Git یا Log	زیاد	دسترسی غیرمجاز به سرویس‌ها	Secret Store، Masking و Rotation فوری
Dependency آسیب‌پذیر	زیاد	ریسک Build یا زنجیره تأمین	Composer/NPM Audit و ارتقای کنترل‌شده
نبود Monitoring	متوسط	کشف دیر هنگام قطعی یا نفوذ	Alert و Uptime، 5xx، DB، Worker، Disk
Log بدون Retention	متوسط	پر شدن دیسک یا نگهداری بیش‌ازحد داده	Rotation، سطح Log مناسب و سیاست نگهداری
انتشار Commit اشتباه	زیاد	بازگشت باگ یا حذف Fix	Deploy با SHA مشخص، Review و ثبت REVISION
نبود UAT واقعی	زیاد	شکست Flowهای واقعی با وجود تست سبز	UAT نقش‌محور و مسیر QR/OTP در Staging

یافته فعلی وابستگی: Composer هیچ Advisory شناخته‌شده‌ای گزارش نکرد، اما NPM یک مورد High در PostCSS نسخه فعلی زنجیره Build گزارش می‌کند. پیش از Release باید dependency به‌صورت کنترل‌شده به‌روزرسانی و همه Gate‌ها دوباره اجرا شوند.

عملیات ضروری روزمره پس از انتشار

روزانه

سلامت up/، خطاهای 5xx، صف ناموفق، فضای دیسک و موفقیت Backup بررسی شود.

هفتگی

Logها، Queryهای کند، مصرف منابع و Audit رخدادها مدیریت می‌شود.

ماهانه

Patch امنیتی، Audit وابستگی و آزمایش Restore یک Backup انجام شود.

قبل از هر Release

تست کامل، Backup، Build معتبر، Migration Review و Rollback Plan تأیید شود.

۷. چک‌لیست نهایی پیش از تصمیم به انتشار

☐ OTP واقعی با سناریوی خطا آزمایش شده است.	☐ Commit نهایی Review و در main ادغام شده است.
☐ Queue و Worker تحت Process Manager هستند.	☐ Staging مشابه Production آماده است.
☐ Storage پایدار و دسترسی فایل صحیح است.	☐ PHP 8.4.1+ و همه Extension ها تأیید شده‌اند.
☐ Mail، Log و Monitoring عملیاتی‌اند.	☐ PostgreSQL واقعی Migration و Test شده است.
☐ Composer Audit و NPM Audit قابل قبول‌اند.	☐ Backup معتبر و Restore Drill موفق وجود دارد.
☐ Quality Gate و Build دوباره سبز شده‌اند.	☐ دامنه، DNS و HTTPS معتبر فعال‌اند.
☐ UAT مسیر QR تا Dashboard موفق است.	☐ APP_DEBUG خاموش و APP_KEY امن است.
☐ Health Check و Rollback واقعی تمرین شده‌اند.	☐ Cookie امن و Session پایدار تنظیم شده است.

تصمیم پیشنهادی

اکسپلوریا قابل استقرار روی سرور واقعی است. وضعیت فعلی Codebase برای ادامه توسعه و آماده‌سازی Staging مناسب است؛ ولی انتشار عمومی باید پس از رفع مورد PostCSS، انتقال به PostgreSQL، اتصال OTP واقعی و موفقیت Gate‌های Production انجام شود.

مراجع داخلی این سند

composer.json	الزامات Runtime
.env.example	متغیرهای محیطی
scripts/deploy-staging.sh	فرآیند استقرار Staging
demo-readiness و production-readiness	Gate‌های پروژه
EXPLORIA AI Development Framework v1.0	چارچوب فنی

دامنه سند: راهنمای برنامه‌ریزی و ارزیابی ریسک؛ بدون اجرای Deployment یا تغییر محیط فعلی.